

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ**

Факультет прикладной информатики

Образовательная программа Мобильные и сетевые технологии

Направление подготовки 09.03.03 Мобильные и сетевые технологии

О Т Ч Е Т

Лабораторная работа №6

Тема работы: «Работа с БД в СУБД MongoDB»

Обучающийся: Федорова Мария Витальевна, К3239

Поток: ПиРБД К24 МСТ 1.2

Преподаватель: Говорова М.М.

Санкт-Петербург,
2026

1. Цель работы

Овладеть практическими навыками работы с базами данных в СУБД MongoDB: создания и выбора базы данных, выполнения операций добавления, поиска, изменения и удаления документов, работы с вложенными объектами и массивами, использования JavaScript в консоли MongoDB, применения курсоров, агрегатных операций и ссылок между коллекциями.

2. Практическое задание

В ходе выполнения лабораторной работы требовалось:

1. установить и запустить сервер MongoDB;
2. подключиться к серверу с помощью MongoDB Compass и `mongosh`;
3. создать и выбрать рабочую базу данных;
4. выполнить CRUD-операции над коллекциями базы данных;
5. выполнить запросы с условиями, сортировкой, ограничением выборки и проекциями;
6. выполнить операции с вложенными объектами и массивами;
7. использовать JavaScript-функции и курсоры в консоли MongoDB;
8. выполнить агрегатные запросы;
9. показать работу ссылок между коллекциями и объединения данных с помощью `$lookup`.

3. Краткие сведения о выполнении части 6.1

На первом этапе работы была выполнена установка MongoDB Community Edition, после чего сервер MongoDB был запущен локально. Для подключения использовался локальный адрес `localhost:27018`. Также была проверена возможность подключения к серверу через MongoDB Compass и консоль `mongosh`.

Далее основная часть лабораторной работы выполнялась в рамках раздела 6.2, посвящённого практической работе с коллекциями и документами MongoDB.

4. Выполнение работы

4.1. Подключение к серверу и выбор рабочей базы данных

После запуска сервера было выполнено подключение к MongoDB из консоли `mongosh`. Далее были проверены основные команды объекта `db`, состояние текущей базы данных и выполнен переход к рабочей базе данных `learn`.

```

learn> use learn
already on db learn
learn> db.unicorns.insertMany([
|   {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
|   {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
|   {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
|   {name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
|   {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
|   {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
|   {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
|   {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
|   {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
|   {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
|   {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0a0eaddfeb861ba0cb5fc5'),
    '1': ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
    '2': ObjectId('6a0a0eaddfeb861ba0cb5fc7'),
    '3': ObjectId('6a0a0eaddfeb861ba0cb5fc8'),
    '4': ObjectId('6a0a0eaddfeb861ba0cb5fc9'),
    '5': ObjectId('6a0a0eaddfeb861ba0cb5fca'),
    '6': ObjectId('6a0a0eaddfeb861ba0cb5fcb'),
    '7': ObjectId('6a0a0eaddfeb861ba0cb5fcc'),
    '8': ObjectId('6a0a0eaddfeb861ba0cb5fcd'),
    '9': ObjectId('6a0a0eaddfeb861ba0cb5fce'),
    '10': ObjectId('6a0a0eaddfeb861ba0cb5fcf')
  }
}
learn> var uni = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}

```

Рис. 1. Подключение к серверу MongoDB из консоли mongosh

```

learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc5'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc7'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc8'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc9'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fca'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcb'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcc'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
]

```

Рис. 2. Получение справки по объекту db

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a0a0ebddf861ba0cb5fd0'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc5'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcb'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fce'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcc'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc8'),
    name: 'Rooooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc7'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
learn>
```

Рис. 3. Просмотр статистики текущей базы данных

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fca'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcd'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn>
```

Рис. 4. Переход к рабочей базе данных learn

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  _id: ObjectId('6a0a0eaddfeb861ba0cb5fc9'),
  name: 'Selma',
  loves: [ 'apple', 'carrot', 'chocolate' ],
  weight: 550,
  gender: 'f',
  vampires: 80
},
{
  _id: ObjectId('6a0a0eaddfeb861ba0cb5fcf'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
}
learn>
```

Рис. 5. Просмотр списка доступных баз данных

В результате была выбрана рабочая база данных `learn`, в которой далее создавались и изменялись коллекции.

4.2. Создание коллекции `unicorns` и заполнение рабочими данными

Для выполнения основной части лабораторной работы была использована коллекция `unicorns`. В неё были добавлены документы, описывающие единорогов: имя, предпочтения в еде, вес, пол и количество побежденных вампиров.

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn>
```

Рис. 6. Добавление первого документа в коллекцию `unicorns`

```
learn> db.unicorns.find(
  { gender: 'm' },
  { _id: 0, loves: 0, gender: 0 }
).sort({name: 1})
[
  { name: 'Dunx', weight: 704, vampires: 165 },
  { name: 'Horny', weight: 600, vampires: 63 },
  { name: 'Kenny', weight: 690, vampires: 39 },
  { name: 'Pilot', weight: 650, vampires: 54 },
  { name: 'Raleigh', weight: 421, vampires: 2 },
  { name: 'Rooooooodles', weight: 575, vampires: 99 },
  { name: 'Unicrom', weight: 984, vampires: 182 }
]
learn>
```

Рис. 7. Появление коллекции `unicorns` в базе данных

```

learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a0a0ebddfeb861ba0cb5fd0'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcf'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fce'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcd'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcc'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcb'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fca'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc9'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  }
]

```

Рис. 8. Массовая вставка документов в коллекцию unicorns

```

learn> db.unicorns.find(
  { },
  { _id: 0, name: 1, loves: {$slice: 1} }
)
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn>

```

Рис. 9. Проверка содержимого коллекции unicorns

После выполнения вставки коллекция unicorns была заполнена набором докумен-

тов, который затем использовался для выполнения запросов и модификации данных.

4.3. Базовые запросы: фильтрация, сортировка, ограничение выборки и проекции

На следующем этапе были выполнены базовые операции выборки документов с использованием фильтрации, сортировки, ограничения количества результатов, выбора одного документа и проекций.

```
learn> db.unicorns.find({weight: {$gt: 700}})
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc7'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fca'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0a0ebddfeb861ba0cb5fd0'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn> █
```

Рис. 10. Выборка самцов с сортировкой по имени

```

learn> db.unicorns.find({weight: {$gte: 500, $lte: 700}})
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc5'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc8'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc9'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcb'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcd'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fce'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcf'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>

```

Рис. 11. Выборка самок с сортировкой и ограничением количества документов

```

learn> db.unicorns.find({loves: {$all: ['apple', 'watermelon']}})
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcd'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fce'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  }
]
learn>

```

Рис. 12. Поиск самок, которые любят carrot, и выбор одного документа

```
learn> db.unicorns.find({vampires: {$exists: true}})
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc5'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc7'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc8'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc9'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fca'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcb'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcc'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],

```

Рис. 13. Проекция выбранных полей без вывода `_id`

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcf'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> █
```

Рис. 14. Сортировка документов по полю `$natural`

```
learn> db.unicorns.find({gender: 'f', vampires: {$gt: 50}})
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc9'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  }
]
learn> █
```

Рис. 15. Ограничение массива в проекции с помощью `$slice`

Выполненные запросы показали, что MongoDB позволяет удобно получать как все поля документа, так и только необходимые данные с помощью проекций.

4.4. Запросы с условиями и операторами

Далее были выполнены запросы с операторами сравнения, принадлежности, проверки существования полей и работы с массивами.

```
learn> db.unicorns.find({loves: {$in: ['lemon', 'papaya']}})
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc5'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcb'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn>
```

Рис. 16. Выборка документов с весом больше 700

```
learn> db.unicorns.find({loves: {$ne: 'apple'}})
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc5'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc7'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fca'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcb'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcf'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0a0ebddfeb861ba0cb5fd0'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn>
```

Рис. 17. Выборка документов в диапазоне значений веса

```
learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: [""],
    mayor: { name: "Jim Wehrle" }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: { name: "Sam Adams", party: "D" }
  }
])

db.towns.find(
  { "mayor.party": "I" },
  { _id: 0, name: 1, mayor: 1 }
)

db.towns.find(
  { "mayor.party": { $exists: false } },
  { _id: 0, name: 1, mayor: 1 }
)
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn>
```

Рис. 18. Поиск документов, массив loves которых содержит несколько значений

```
learn> function showMaleUnicorns() {
  return db.unicorns.find({ gender: "m" })
}

var cursor = db.unicorns.find({ gender: "m" }).sort({ name: 1 }).limit(2)
cursor.forEach(function(obj) {
  print(obj.name)
})
Dunx
Horny
learn>
```

Рис. 19. Проверка существования поля vampires

```
learn> db.unicorns.find(
  {
    gender: 'f',
    weight: { $lt: 500, $lte: 400 }
  })
[ { _id: 1, name: 'Dunx', weight: 450, gender: 'f' } ]
(node:5456) [MONGODB DRIVER] Warning: cursor.count is deprecated and will be removed in the next major version, please use 'collection.estimatedDocumentCount' or 'collection.countDocuments' instead
(Use 'node --trace-warnings ...' to show where the warning was created)
learn>
```

Рис. 20. Поиск документов, в которых поле vampires отсутствует

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn>
```

Рис. 21. Комбинированный запрос по полу и количеству вампиров

```
learn> db.unicorns.aggregate([
  { $group: { _id: "$gender", count: { $sum: 1 } } }
])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn>
```

Рис. 22. Поиск по оператору \$in в массиве предпочтений

```
learn> db.unicorns.insertOne({
  |   name: "Barny",
  |   loves: ["grape"],
  |   weight: 340,
  |   gender: "m"
  | })
  |
  | db.unicorns.find({ name: "Barny" })
  | [
  |   {
  |     _id: ObjectId('6a0a193ddfeb861ba0cb5fd4'),
  |     name: 'Barny',
  |     loves: [ 'grape' ],
  |     weight: 340,
  |     gender: 'm'
  |   }
  | ]
learn> █
```

Рис. 23. Поиск по оператору \$ne для значения массива

Использование операторов \$gt, \$gte, \$lte, \$all, \$exists, \$in и \$ne позволило выполнить более сложную фильтрацию документов по различным условиям.

4.5. Работа с вложенными объектами на примере коллекции towns

Для демонстрации работы со вложенными документами была создана коллекция towns, документы которой содержат сведения о городе и вложенный объект mayor.

```
learn> db.unicorns.updateOne(
  | { name: "Ayna" },
  | { $set: { weight: 800, vampires: 51 } }
  | )
  |
  | db.unicorns.find({ name: "Ayna" })
  | [
  |   {
  |     _id: ObjectId('6a0a0eaddfeb861ba0cb5fca'),
  |     name: 'Ayna',
  |     loves: [ 'strawberry', 'lemon' ],
  |     weight: 800,
  |     gender: 'f',
  |     vampires: 51
  |   }
  | ]
learn> █
```

Рис. 24. Создание коллекции towns и поиск городов по вложенному полю

На данном этапе были созданы документы с вложенным объектом mayor, после чего выполнены запросы по полю mayor.party и по отсутствию этого поля.

4.6. Использование JavaScript и курсоров

MongoDB позволяет использовать конструкции JavaScript непосредственно в консоли mongosh. Для проверки этой возможности была создана функция вывода самцов единорогов, а также курсор с сортировкой и ограничением размера выборки.

```
learn> db.unicorns.updateOne(
  | { name: "Raleigh" },
  | { $set: { loves: ["apple", "sugar", "redbull"] } }
  | )
  |
  | db.unicorns.find({ name: "Raleigh" })
  | [
  |   {
  |     _id: ObjectId('6a0a0eaddfeb861ba0cb5fcc'),
  |     name: 'Raleigh',
  |     loves: [ 'apple', 'sugar', 'redbull' ],
  |     weight: 421,
  |     gender: 'm',
  |     vampires: 2
  |   }
  | ]
learn> █
```

Рис. 25. Использование JavaScript-функции и курсора с forEach

```

learn> db.unicorns.updateMany(
|   { gender: "m" },
|   { $inc: { vampires: 5 } }
| )
|
| db.unicorns.find({ gender: "m" })
|
| [
|   {
|     _id: ObjectId('6a0a0eaddfeb861ba0cb5fc5'),
|     name: 'Horny',
|     loves: [ 'carrot', 'papaya' ],
|     weight: 600,
|     gender: 'm',
|     vampires: 68
|   },
|   {
|     _id: ObjectId('6a0a0eaddfeb861ba0cb5fc7'),
|     name: 'Unicrom',
|     loves: [ 'energon', 'redbull' ],
|     weight: 984,
|     gender: 'm',
|     vampires: 187
|   },
|   {
|     _id: ObjectId('6a0a0eaddfeb861ba0cb5fc8'),
|     name: 'Rooooooodles',
|     loves: [ 'apple' ],
|     weight: 575,
|     gender: 'm',
|     vampires: 104
|   },
|   {
|     _id: ObjectId('6a0a0eaddfeb861ba0cb5fcb'),
|     name: 'Kenny',
|     loves: [ 'grape', 'lemon' ],
|     weight: 690,
|     gender: 'm',
|     vampires: 44
|   },
|   {
|     _id: ObjectId('6a0a0eaddfeb861ba0cb5fcc'),
|     name: 'Raleigh',
|     loves: [ 'apple', 'sugar', 'redbull' ],
|     weight: 421,
|     gender: 'm',
|     vampires: 7
|   },
|   {
|     _id: ObjectId('6a0a0eaddfeb861ba0cb5fce'),
|     name: 'Pilot',
|     loves: [ 'apple', 'watermelon' ],
|     weight: 650,
|     gender: 'm',
|     vampires: 59
|   },
|   {
|     _id: ObjectId('6a0a0ebddfeb861ba0cb5fd0'),
|     name: 'Dunx',
|     loves: [ 'grape', 'watermelon' ],
|     weight: 704,

```

Рис. 26. Подсчёт количества документов через курсор

```

learn> db.towns.updateOne(
|   { name: "Portland" },
|   { $unset: { "mayor.party": 1 } }
| )
|
| db.towns.find({ name: "Portland" })
|
| [
|   {
|     _id: ObjectId('6a0a1866dfef861ba0cb5fd3'),
|     name: 'Portland',
|     population: 528000,
|     last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
|     famous_for: [ 'beer', 'food' ],
|     mayor: { name: 'Sam Adams' }
|   }
| ]
learn>

```

Рис. 27. Получение множества уникальных значений с помощью distinct

Курсоры позволили перебрать результаты выборки, а использование JavaScript в консоли MongoDB показало возможность программной обработки документов.

4.7. Агрегация данных

Для обработки набора документов были выполнены агрегатные запросы с группировкой по полу единорогов, подсчётом количества документов, нахождением среднего веса и максимального значения числа побежденных вампиров.

```
learn> db.unicorns.updateOne(
  { name: "Pilot" },
  { $push: { loves: "chocolate" } }
)
[
  db.unicorns.find({ name: "Pilot" })
]
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fce'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> █
```

Рис. 28. Агрегатные запросы по коллекции unicorns

С помощью конвейера агрегации были получены сводные данные по группам документов, что подтверждает возможность аналитической обработки данных в MongoDB.

4.8. Изменение документов

Следующим этапом были выполнены операции обновления документов с использованием операторов \$set, \$inc, \$unset, \$push и \$addToSet.

```
learn> db.unicorns.updateOne(
  { name: "Aurora" },
  { $addToSet: { loves: { $each: ["sugar", "lemon"] } } }
)
[
  db.unicorns.find({ name: "Aurora" })
]
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> █
```

Рис. 29. Изменение одного документа в коллекции unicorns

```
learn> db.towns.drop()
true
learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: { name: "Jim Wehrle" }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    name: "Portland",
    population: 520000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: { name: "Sam Adams", party: "D" }
  }
])
[
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0a1bd8dfcb861ba0cb5fd5'),
    '1': ObjectId('6a0a1bd8dfcb861ba0cb5fd6'),
    '2': ObjectId('6a0a1bd8dfcb861ba0cb5fd7')
  }
]
learn> █
```

Рис. 30. Изменение массива предпочтений у документа Raleigh

```

learn> db.towns.remove({ "mayor.party": { $exists: false } })
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a0a1bd8dfcb861ba0cb5fd6'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0a1bd8dfcb861ba0cb5fd7'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> db.towns.remove({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
learn>

```

Рис. 31. Массовое изменение документов в коллекции unicorns

```

learn> db.habitats.insertMany([
  { _id: "H1", name: "Rainbow Falls", climate: "humid" },
  { _id: "H2", name: "Candy Mountain", climate: "mild" },
  { _id: "H3", name: "Apple Forest", climate: "cool" }
])
{
  acknowledged: true,
  insertedIds: { '0': 'H1', '1': 'H2', '2': 'H3' }
}
learn> db.unicorns.updateOne(
  { name: "Aurora" },
  { $set: { habitat: "H1" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  { name: "Pilot" },
  { $set: { habitat: "H2" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  { name: "Nimue" },
  { $set: { habitat: "H3" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find(
  { habitat: { $exists: true } },
  { _id: 0, name: 1, habitat: 1 }
)
[
  { name: 'Aurora', habitat: 'H1' },
  { name: 'Pilot', habitat: 'H2' },
  { name: 'Nimue', habitat: 'H3' }
]
learn> db.habitats.find()
[
  { _id: 'H1', name: 'Rainbow Falls', climate: 'humid' },
  { _id: 'H2', name: 'Candy Mountain', climate: 'mild' },
  { _id: 'H3', name: 'Apple Forest', climate: 'cool' }
]
learn>

```

Рис. 32. Удаление поля во вложенном документе коллекции towns

```

learn> db.foods.insertMany([
  { _id: "F1", name: "apple", color: "red" },
  { _id: "F2", name: "grape", color: "green" },
  { _id: "F3", name: "carrot", color: "orange" },
  { _id: "F4", name: "watermelon", color: "green" }
])
{
  acknowledged: true,
  insertedIds: { '0': 'F1', '1': 'F2', '2': 'F3', '3': 'F4' }
}
learn> db.unicorns.updateOne(
  { name: "Aurora" },
  { $set: { favorite_food_ref: "F3" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  { name: "Pilot" },
  { $set: { favorite_food_ref: "F4" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  { name: "Dunx" },
  { $set: { favorite_food_ref: "F2" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find(
  { favorite_food_ref: { $exists: true } },
  { _id: 0, name: 1, favorite_food_ref: 1 }
)
[
  { name: 'Aurora', favorite_food_ref: 'F3' },
  { name: 'Pilot', favorite_food_ref: 'F4' },
  { name: 'Dunx', favorite_food_ref: 'F2' }
]
learn> db.foods.find()
[
  { _id: 'F1', name: 'apple', color: 'red' },
  { _id: 'F2', name: 'grape', color: 'green' },
  { _id: 'F3', name: 'carrot', color: 'orange' },
  { _id: 'F4', name: 'watermelon', color: 'green' }
]
learn>

```

Рис. 33. Добавление нового значения в массив через \$push

```

learn> db.unicorns.aggregate([
  { $group: { _id: "$gender", count: { $sum: 1 } } }
])
[ { _id: 'm', count: 8 }, { _id: 'f', count: 5 } ]
learn> db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      avg_weight: { $avg: "$weight" }
    }
  }
])
[ { _id: 'm', avg_weight: 620.5 }, { _id: 'f', avg_weight: 588.2 } ]
learn> db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      max_vampires: { $max: "$vampires" }
    }
  }
])
[ { _id: 'm', max_vampires: 187 }, { _id: 'f', max_vampires: 80 } ]
learn> db.unicorns.aggregate([
  { $group: { _id: "$gender", count: { $sum: 1 } } },
  { $sort: { _id: 1 } }
])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 8 } ]
learn>

```

Рис. 34. Добавление уникальных значений в массив через \$addToSet

Выполненные команды показали, что MongoDB позволяет изменять как простые поля, так и массивы, а также вложенные атрибуты документов.

4.9. Удаление документов и коллекций

После выполнения операций изменения были проверены операции удаления отдельных документов, групп документов и коллекций.

```
learn> db.unicorns.aggregate([
  {
    $match: { favorite_food_ref: { $exists: true } }
  },
  {
    $lookup: {
      from: "foods",
      localField: "favorite_food_ref",
      foreignField: "_id",
      as: "favorite_food"
    }
  }
])
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
    habitat: 'H1',
    favorite_food_ref: 'F3',
    favorite_food: [ { _id: 'F3', name: 'carrot', color: 'orange' } ]
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fce'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59,
    habitat: 'H2',
    favorite_food_ref: 'F4',
    favorite_food: [ { _id: 'F4', name: 'watermelon', color: 'green' } ]
  },
  {
    _id: ObjectId('6a0a0ebddfeb861ba0cb5fd0'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 170,
    favorite_food_ref: 'F2',
    favorite_food: [ { _id: 'F2', name: 'grape', color: 'green' } ]
  }
]
learn>
```

Рис. 35. Удаление документов из коллекции **unicorns**

```
learn> db.unicorns.aggregate([
  {
    $match: { favorite_food_ref: { $exists: true } }
  },
  {
    $lookup: {
      from: "foods",
      localField: "favorite_food_ref",
      foreignField: "_id",
      as: "favorite_food"
    }
  },
  {
    $project: {
      _id: 0,
      name: 1,
      favorite_food_ref: 1,
      favorite_food: 1
    }
  }
])
[
  {
    name: 'Aurora',
    favorite_food_ref: 'F3',
    favorite_food: [ { _id: 'F3', name: 'carrot', color: 'orange' } ]
  },
  {
    name: 'Pilot',
    favorite_food_ref: 'F4',
    favorite_food: [ { _id: 'F4', name: 'watermelon', color: 'green' } ]
  },
  {
    name: 'Dunx',
    favorite_food_ref: 'F2',
    favorite_food: [ { _id: 'F2', name: 'grape', color: 'green' } ]
  }
]
learn>
```

Рис. 36. Удаление коллекции **foods** и проверка существующих коллекций

В результате были удалены отдельные документы коллекции **unicorns**, а также удалена коллекция **foods**. При этом остальные коллекции базы данных сохранились.

4.10. Ссылки между коллекциями

Для демонстрации логических связей между коллекциями были созданы дополнительные коллекции `habitats` и `foods`. Далее в документах коллекции `unicorns` были добавлены ссылки на место обитания и любимую еду.

```
learn> db.unicorns.aggregate([
  {
    $match: { habitat: { $exists: true } }
  },
  {
    $lookup: {
      from: "habitats",
      localField: "habitat",
      foreignField: "_id",
      as: "habitat_info"
    }
  }
])
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc6'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
    habitat: 'H1',
    favorite_food_ref: 'F3',
    habitat_info: [ { _id: 'H1', name: 'Rainbow Falls', climate: 'humid' } ]
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fce'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59,
    habitat: 'H2',
    favorite_food_ref: 'F4',
    habitat_info: [ { _id: 'H2', name: 'Candy Mountain', climate: 'mild' } ]
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcf'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f',
    habitat: 'H3',
    habitat_info: [ { _id: 'H3', name: 'Apple Forest', climate: 'cool' } ]
  }
]
learn>
```

Рис. 37. Создание коллекции `habitats` и добавление ссылок на места обитания

```
learn> db.unicorns.aggregate([
  {
    $match: { habitat: { $exists: true } }
  },
  {
    $lookup: {
      from: "habitats",
      localField: "habitat",
      foreignField: "_id",
      as: "habitat_info"
    }
  },
  {
    $project: {
      _id: 0,
      name: 1,
      habitat: 1,
      habitat_info: 1
    }
  }
])
[
  {
    name: 'Aurora',
    habitat: 'H1',
    habitat_info: [ { _id: 'H1', name: 'Rainbow Falls', climate: 'humid' } ]
  },
  {
    name: 'Pilot',
    habitat: 'H2',
    habitat_info: [ { _id: 'H2', name: 'Candy Mountain', climate: 'mild' } ]
  },
  {
    name: 'Nimue',
    habitat: 'H3',
    habitat_info: [ { _id: 'H3', name: 'Apple Forest', climate: 'cool' } ]
  }
]
learn>
```

Рис. 38. Создание коллекции `foods` и добавление ссылок на любимую еду

В результате в документах коллекции `unicorns` появились поля-ссылки, содержащие идентификаторы связанных документов из других коллекций.

4.11. Объединение данных из коллекций с помощью \$lookup

После создания ссылок были выполнены агрегатные запросы с использованием оператора \$lookup, который позволяет объединять данные из разных коллекций по совпадающим полям.

```
learn> db.unicorns.deleteOne({ name: "Barney" })
[| db.unicorns.find({ name: "Barney" })

learn> db.unicorns.deleteMany({ gender: "f", vampires: { $lt: 50 } })
[| db.unicorns.find({ gender: "f" })
[
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fc9'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fca'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  },
  {
    _id: ObjectId('6a0a0eaddfeb861ba0cb5fcf'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f',
    habitat: 'H3'
  }
]
```

Рис. 39. Объединение unicorns и foods с помощью \$lookup

```
[learn> db.foods.drop()
true
[learn> show collections
habitats
towns
unicorns
[learn> db.habitats.find()
[
  { _id: 'H1', name: 'Rainbow Falls', climate: 'humid' },
  { _id: 'H2', name: 'Candy Mountain', climate: 'mild' },
  { _id: 'H3', name: 'Apple Forest', climate: 'cool' }
]
learn> █
```

Рис. 40. Объединение unicorns и habitats с помощью \$lookup

В результате выполнения \$lookup к каждому документу коллекции unicorns были добавлены связанные данные из других коллекций. Это показывает, что в MongoDB можно моделировать связи между наборами документов и получать связанные данные в одном запросе.

5. Выводы

В ходе выполнения лабораторной работы был установлен и запущен сервер MongoDB, выполнено подключение к нему с помощью MongoDB Compass и консоли `mongosh`, после чего была выбрана рабочая база данных `learn`.

В основной части лабораторной работы были освоены базовые операции MongoDB: добавление документов в коллекции, поиск документов по различным условиям, сортировка, ограничение выборки, использование проекций, изменение и удаление документов, а также удаление коллекций.

Отдельно были изучены возможности работы со вложенными объектами и массивами, использование JavaScript-функций и курсоров в консоли MongoDB, а также выполнение агрегатных запросов для сводной обработки данных.

Кроме того, были продемонстрированы ссылки между коллекциями и объединение данных с помощью оператора `$lookup`. Это позволило показать, каким образом в MongoDB можно организовывать связи между документами и получать связанные данные в рамках одного запроса.

Таким образом, цель лабораторной работы достигнута: получены практические навыки работы с базами данных в СУБД MongoDB.